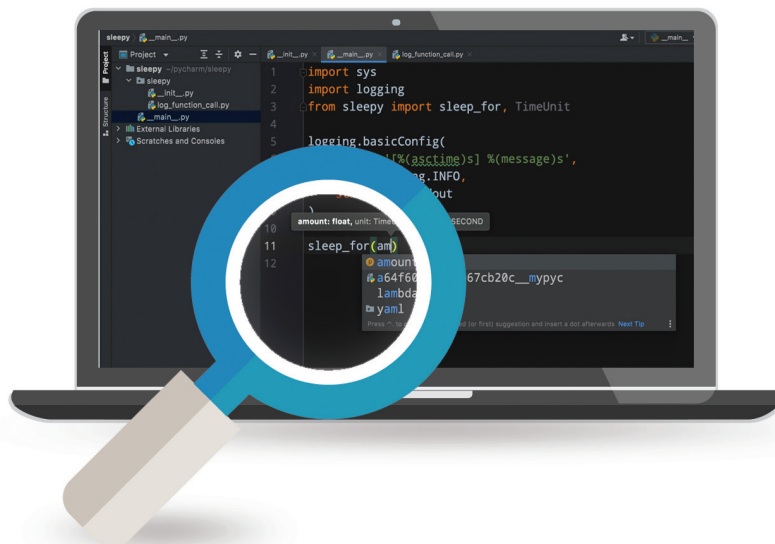


# How to Iron Out IDE Issues with Auto Stub Generation

Automatic stub generation can significantly improve code analysis. Read on to find out why you should consider this, and how to go about it.



A good project takes a desired shape only due to the rigorous effort and time that developers put into it. But all this effort can go straight down the drain if the key components that are essential for development present any problems. During the development of the Toloka-Kit library at Toloka, the team encountered a wide range of such issues with integrated development environment (IDE) support. This article will help you understand the issues faced and how these were dealt with, so you can cross similar hurdles easily.

To put things into perspective, let us first understand what IDE hinting is.

## IDE hinting

Let us say we want to implement a module called `sleepy` (Figure 1). This `sleepy` module will contain a `sleep_for` function between the standard library and time sleep function. Our `sleep_for` function will accept a time unit, so

we will be able to make it sleep for different time periods, say half an hour, a day, or maybe even for weeks.

The team at Toloka implemented this module, and we saw that if we used the `script_for` function anywhere in our code, then the IDE showed us the signature. Upon trying to print the argument name it got autocompleted. If for some reason we passed an argument that was not expected in the signature, then the IDE highlighted this

argument and we would see that we were doing something wrong before running the code.

Let us write a simple decorator called `log_function_call`, as shown in Figure 2. This function does exactly what the name suggests. It will write a line of code in the log stream every time we run the function, and every time the function is complete we will write another message to our log form.

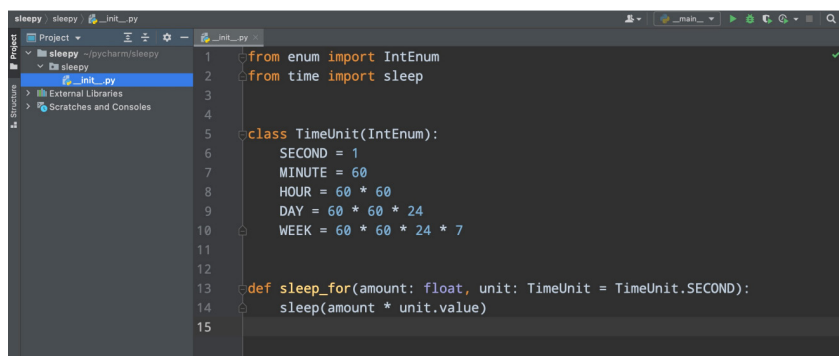


Figure 1: Sleepy module in IDE